# Employment Support Hub – Distributed Systems Project Report

**Student name: Shane Cosgrove**

**Student number: x25115651**

**GitHub link: https://github.com/Shane-Cosgrove/employment-hub**

## 1. Introduction

My purpose in creating this project was to design and implement a distributed system that supports the Sustainable Development Goal (SDG) 8: Decent Work and Economic Growth. The system I developed for this project is called the Employment Support Hub.

The Employment Support Hub is a distributed application that assists job seekers by providing services such as job matching, interview scheduling, and skills recommendation. The application was implemented using gRPC and Protocol Buffers (Proto3) in Node.js.

Important distributed systems concepts I've included in the project as required are listed below including:

- Microservice architecture,
- Remote Procedure Calls (RPC),
- Service discovery and registration,
- Streaming communication,
- Error handling,
- Graphical user interfaces,
- Distributed communication between services.

When developing the app, I ensured there was multiple independent services that communicated through gRPC, and I implemented a central registry service to allow services to register and be discovered dynamically.

## 2. System Architecture

The Employment Support Hub follows a microservices-based distributed architecture.

I designed the system so that it contains the following major components:

- Client Browser GUI,
- GUI Server (Express.js),
- Registry Service,
- Job Matching Service,
- Interview Scheduling Service,
- Skills Recommendation Service.

I've each service operating independently and communicating using gRPC over localhost ports.

### 2.1 GUI Layer

I've designed the system to include a browser-based graphical user interface built using HTML (basic) and JavaScript. The GUI then communicates with an Express.js controller server which acts as the main gateway between the browser and the backend microservices.

The GUI allows users to:

- Test the Job Matching Service,
- Test the Interview Scheduling Service,
- Test the Skills Recommendation Service,
- View JSON outputs directly in the browser.

### 2.2 Registry Service

The Registry Service acts as a naming and discovery service with all backend services registering themselves with the registry when they start.

The registry stores:

- Service name,
- Host,
- Port.

This enables distributed service discovery.

### 2.3 Job Matching Service

I've designed the Job Matching Service to manage the following:

- Job seeker registration,
- Job posting,
- Match calculation.

The service uses server-side streaming to stream matching job results back to the client.

### 2.4 Interview Scheduling Service

The Interview Scheduling Service manages:

- Interview slot creation,
- Booking interviews,
- Viewing interview booking status.

The ensures that interview slots cannot be double-booked.

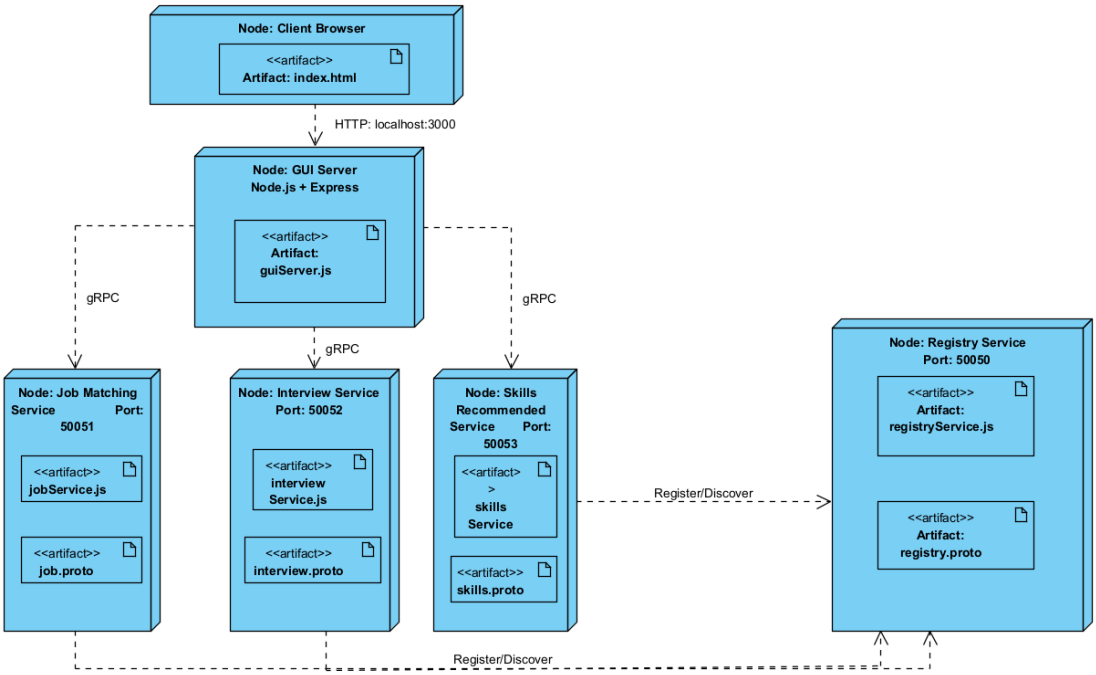
### 2.5 Skills Recommendation Service

The Skills Recommendation Service performs:

- Skill gap analysis,
- Training recommendations.

The service compares a candidate's current skills against job requirements and recommends training courses for missing skills, so they have an increased chance to gain employment.

### 2.6 Architecture Diagram




## 3. Service Definitions

The system uses Protocol Buffers (Proto3) to define all service contracts.

Each service contains the below:

- RPC method definitions
- Request message structures
- Response message structures

### 3.1 Job Service Definition

The Job Service contains three RPC methods:

- RegisterJobSeeker - Registers a new job seeker.
- PostJob - Creates a new job posting.
- FindMatches - Streams matching jobs back to the client.

This RPC uses server-side streaming.

### 3.2 Interview Service Definition

The Interview Service contains the following:

- CreateInterviewSlot - Creates a new interview slot.
- BookInterview - Books an available interview slot.
- GetInterviewStatus - Retrieves interview booking information.

### 3.3 Skills Service Definition

The Skills Service contains the following:

- AnalyzeSkillGap - Identifies missing skills required for a target job.
- RecommendTraining - Provides recommended training courses.

### 3.4 Registry Service Definition

The Registry Service contains:

- RegisterService - Registers services dynamically.
- DiscoverService - Allows services to be discovered.

## 4. Service Implementations

All services were implemented using Node.js and the @grpc/grpc-js library.

### 4.1 Job Matching Service Implementation

The Job Matching Service stores job seekers and jobs using in-memory arrays.

The service performs validation to ensure:

- Required fields are present,
- Skills arrays are not empty,
- Duplicate jobs are prevented.

The service calculates job match percentages by comparing candidate skills against required job skills.

The FindMatches RPC streams results using:

call.write()

and closes the stream using:

call.end()

### 4.2 Interview Scheduling Service Implementation

The Interview Scheduling Service manages interview slots and bookings.

The service checks the following:

- Slot existence,
- Slot availability,
- Candidate validity.

The service prevents interview slots from being booked more than once.

### 4.3 Skills Recommendation Service Implementation

The Skills Recommendation Service compares current candidate skills against required job skills.

Missing skills are identified using JavaScript filtering logic.

Training recommendations are generated dynamically using the missing skills list.

### 4.4 Registry Service Implementation

The Registry Service stores all active services in memory.

When services start, they automatically register themselves using:

RegisterService()

The registry allows dynamic service discovery and demonstrates naming service functionality within distributed systems.

## 5. Naming Services

The Registry Service acts as the naming service for the distributed application.

Each backend service registers itself with:

- Service name,
- Host address,
- Port number.

Examples:

| Service | Port |
|---|---|
| **Registry Service** | **50050** |
| **Job Matching Service** | **50051** |
| **Interview Scheduling Service** | **50052** |
| **Skills Recommendation Service** | **50053** |
| **GUI Server** | **3000** |

The naming service allows distributed services to locate and communicate with each other.

This demonstrates service registration and service discovery principles used in modern distributed systems.

## 6. Error Handling and Advanced Features

The system implements multiple forms of remote error handling. The system also implements client-side streaming and bidirectional streaming RPC communication using gRPC. Client-side streaming was implemented in the Skills Recommendation Service, while bidirectional streaming was implemented in the Job Matching Service.

### 6.1 Validation Errors

Services validate:

- Missing fields,
- Empty arrays,
- Invalid requests.

Invalid requests return gRPC error codes such as:

**INVALID_ARGUMENT**

### 6.2 Duplicate Prevention

The Job Matching Service prevents duplicate jobs from being added.

Duplicate job requests return:

**ALREADY_EXISTS**

### 6.3 Interview Availability Validation

The Interview Scheduling Service prevents double booking.

If a slot has already been booked, the service returns:

**FAILED_PRECONDITION**

### 6.4 Unavailable Service Handling

The GUI correctly reports unavailable services when a backend service is offline.

Example:

**UNAVAILABLE: No connection established**

### 6.5 Streaming RPC

The FindMatches method in the Job Matching Service uses server-side streaming.

This allows multiple matching jobs to be streamed back dynamically.

### 6.6 GUI Integration

A browser-based GUI was implemented to demonstrate distributed communication visually.

The GUI controller communicates with all backend services using gRPC.

## 7. Client Graphical User Interface (GUI)

The project includes a web-based GUI built using:

**HTML**

**JavaScript**

**Express.js**

The GUI provides a central controller for interacting with the distributed system.

The interface includes:

- Test Job Matching Service button
- Test Interview Scheduling Service button
- Test Skills Recommendation Service button

- Output display section

The GUI sends HTTP requests to the Express controller server.

The Express controller then communicates with the backend microservices using gRPC clients.

This demonstrates integration between web technologies and distributed microservices.

### 7.1 GUI Features

The GUI supports:

- Service testing
- Dynamic JSON output
- Error reporting
- Distributed communication

### 7.2 Screenshots

Screenshots at end of doc.

## 8. GitHub link: https://github.com/Shane-Cosgrove/employment-hub

My GitHub repository contains:

- Full source code,
- Proto files,
- Services,
- GUI implementation,
- Architecture diagram,
- Commit history.

Git was used throughout development to demonstrate project progression.

## 9. Conclusion

The Employment Support Hub demonstrates the design and implementation of a distributed system using gRPC and Node.js.

The project implemented multiple independent microservices that communicate remotely using Protocol Buffers and RPC communication.

Key distributed systems concepts demonstrated include:

- Microservices architecture,
- Service discovery,
- Naming services,
- Remote procedure calls,
- Streaming communication,
- Error handling,
- Distributed service coordination.

The project also supports Sustainable Development Goal 8 by providing services that assist job seekers with employment opportunities, interview management, and skill development.

Future improvements could include:

- Database persistence,
- Docker containerization,
- Kubernetes orchestration,
- Authentication and authorization,
- Real-time notifications,
- Cloud deployment.

Overall, the project successfully achieved the objectives of implementing a modern distributed application with multiple interacting services and a graphical user interface.

## Index.html initiation and progress

**Before button implementation:**


# Employment Support Hub

Smart distributed application supporting SDG 8: Decent Work and Economic Growth.

## Available Services

- Job Matching Service
- Interview Scheduling Service
- Skills Recommendation Service
- Service Registry

---

**Button implementation GUI homepage:**


# Employment Support Hub

Smart distributed application supporting SDG 8: Decent Work and Economic Growth.

## Service Controller

Test Job Matching Service | Test Interview Scheduling Service | Test Skills Recommendation Service

## Output

```
Results will appear here...
```

**Output for each button:**

**Test Job Matching Service with error handling example:**

# Employment Support Hub

Smart distributed application supporting SDG 8: Decent Work and Economic Growth.

## Service Controller

Test Job Matching Service | Test Interview Scheduling Service | Test Skills Recommendation Service

## Output


```
{
  "message": "Job seeker Shane registered successfully",
  "matches": [
    {
      "job_id": "J20",
      "title": "Customer Service Representative",
      "match_score": 100
    }
  ]
}
```


# Employment Support Hub

Smart distributed application supporting SDG 8: Decent Work and Economic Growth.

## Service Controller

Test Job Matching Service | Test Interview Scheduling Service | Test Skills Recommendation Service

## Output


```
{
  "error": "6 ALREADY_EXISTS: Job already exists"
}
```

**Test Interview Scheduling Service with error handling example:**

# Employment Support Hub

Smart distributed application supporting SDG 8: Decent Work and Economic Growth.

## Service Controller

Test Job Matching Service | Test Interview Scheduling Service | Test Skills Recommendation Service

## Output


```
{
  "slot": "Interview slot S101 created successfully",
  "booking": {
    "booking_id": "B1",
    "status": "Booked",
    "message": "Interview slot booked successfully"
  },
  "status": {
    "booking_id": "B1",
    "slot_id": "S101",
    "candidate_id": "JS101",
    "status": "Booked"
  }
}
```



# Employment Support Hub

Smart distributed application supporting SDG 8: Decent Work and Economic Growth.

## Service Controller

Test Job Matching Service | Test Interview Scheduling Service | Test Skills Recommendation Service


## Output


```
{
  "error": "9 FAILED_PRECONDITION: Interview slot is not available"
}
```

**Test Skills Recommendation Service**

# Employment Support Hub

Smart distributed application supporting SDG 8: Decent Work and Economic Growth.

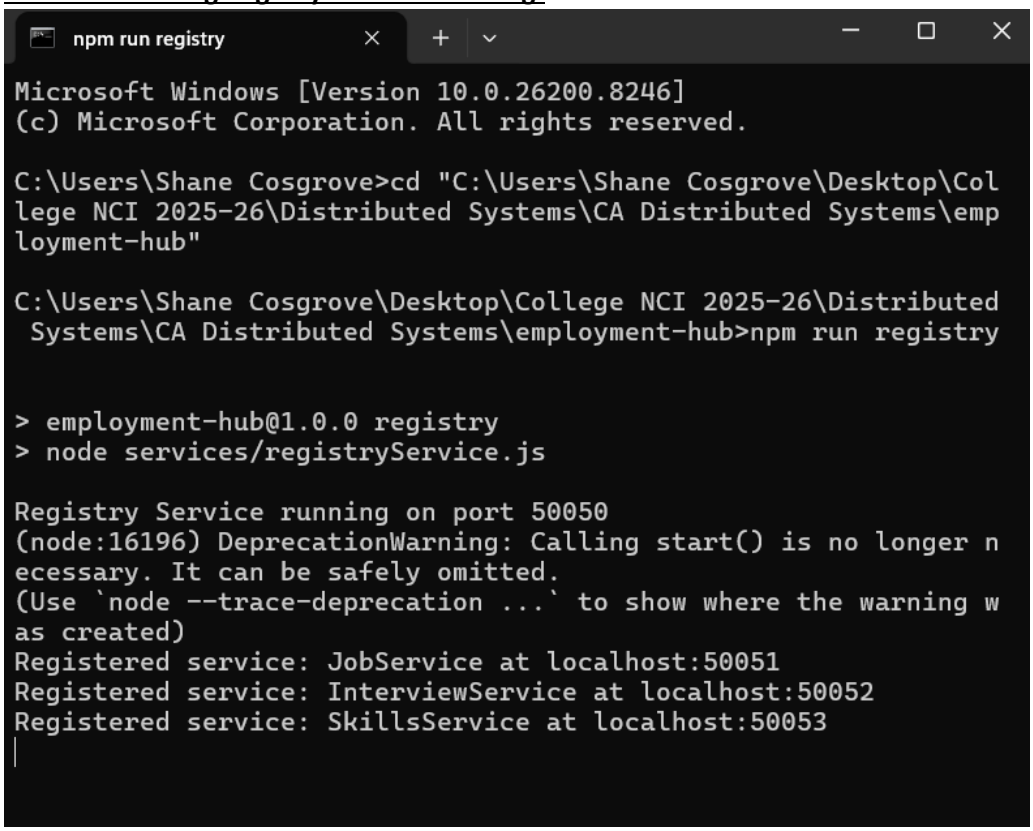
## Service Controller

Test Job Matching Service | Test Interview Scheduling Service | Test Skills Recommendation Service

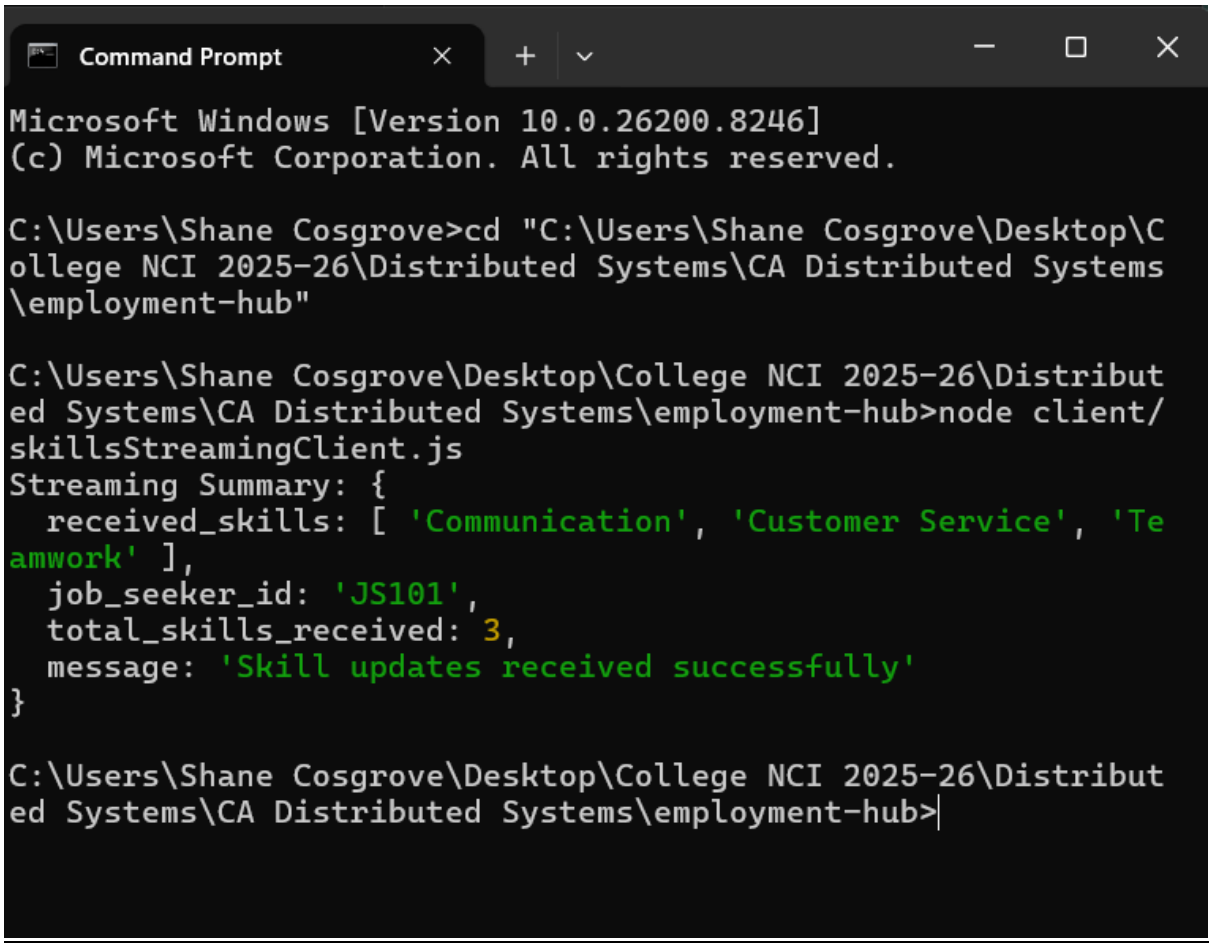
## Output


```
{
  "skill_gap": {
    "missing_skills": [
      "POS Systems",
      "Teamwork"
    ],
    "message": "Skill gap analysis completed successfully"
  },
  "training": {
    "recommended_courses": [
      "POS Systems Fundamentals Training",
      "Teamwork Fundamentals Training"
    ],
    "message": "Training recommendations generated successfully"
  }
}
```


**Terminal showing Registry Services Running:**



**Client-Side Streaming RPC Output**




**Bidirectional Streaming RPC Output**